

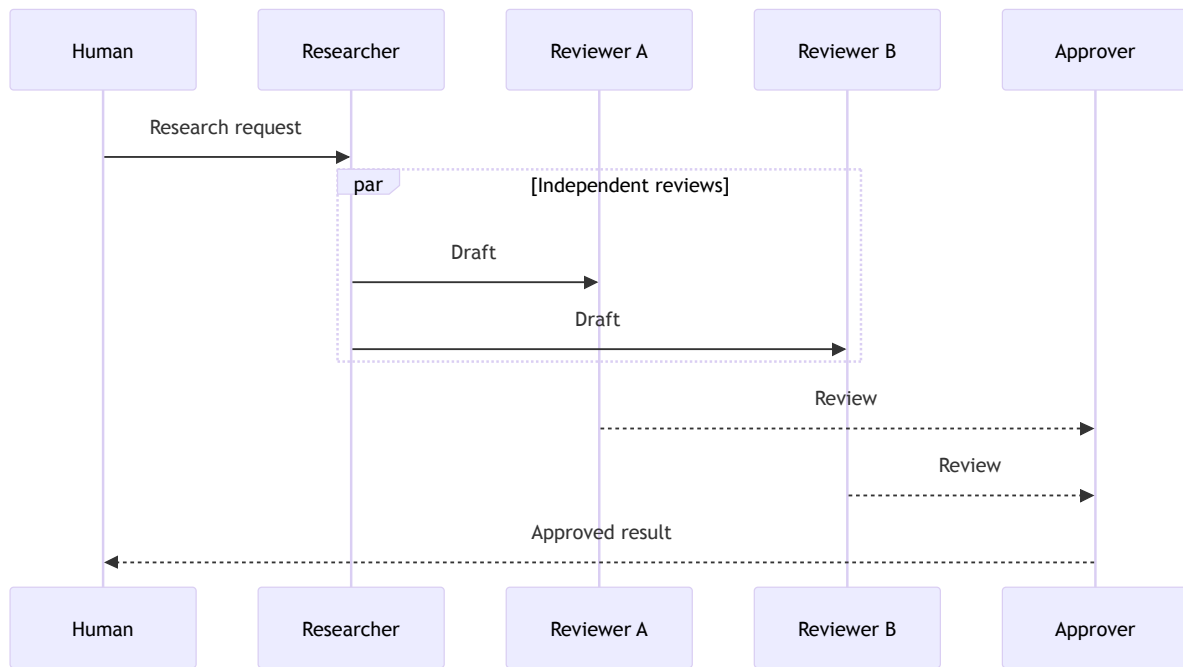
# Кто управляет разговором агентов?

Сценарии, протоколы и runtime-ы мультиагентных  
платформ

Конкурентная карта для Reagent

# Одна задача — пять разных артефактов

Нужно выполнить один сценарий: **research** → **parallel review** → **approval** → **publish**.



LangGraph

AutoGen

CrewAI

JADE

Moise

# Настоящий конкурент = descriptor + runtime

## Descriptor

Как зафиксированы роли, порядок, branching, delegation и допустимые коммуникации?

## Runtime

Кто исполняет, маршрутизирует, сохраняет состояние, восстанавливает и наблюдает swarm?

MCP/A2A без orchestration и single-agent SDK без scenario model — смежные слои, не прямые конкуренты.

# Три линии развития

## Workflow / graph

LangGraph  
Microsoft Agent Framework  
Dapr Agents  
Google ADK  
CrewAI Flows

Primary artifact: application execution graph

## Teams / swarms

AutoGen  
CrewAI Crews  
OpenAI Agents SDK  
CAMEL / AgentScope

Primary artifact: team policy and delegation

## Protocols / organizations

JADE + FIPA  
JaCaMo + Moise  
Scribble / MPST  
Choral

Primary artifact: legal interaction or organization

LLM frameworks дали новую волну автономии. Protocol systems напоминают, что coordination — отдельная инженерная дисциплина.

# По каким осям сравниваем

## 1. Source of truth

код, graph, config, SOP или protocol artifact

## 3. Communication semantics

messages/roles enforced или просто function outputs

## 5. Durability

checkpoint, resume, queue, crash recovery

## 7. Verification

type checks, graph validation, conformance, formal model

## 2. Global view

виден ли весь разговор или только локальные handlers

## 4. Control

deterministic edges, model routing, handoff, norms

## 6. Distribution

один process, workers, multi-node agent runtime

## 8. Operability

deploy, tracing, HITL, debugging, ecosystem

«Есть multi-agent» — слишком слабый критерий. Важно, что именно runtime обещает сохранить и enforce.

# Карта рынка

Качественная карта: не benchmark и не ranking adoption

## Сильный production runtime

LangGraph · Microsoft Agent Framework · Dapr Agents

Durability и workflow control сильнее protocol semantics

## Богатые multi-agent patterns

AutoGen · Google ADK · OpenAI Agents SDK

Teams, handoffs, managers, loops; choreography обычно code-first

## Явный interaction / organization contract

JADE/FIPA · JaCaMo/Moise · MPST/Scribble

Сильные semantics; слабее современный LLM/cloud DX

## Целевая позиция Reagent

Global choreography + role runtime + agent integrations

Позиция убедительна только после hardening durability и enforcement

# LangGraph и Microsoft Agent Framework

## Самые сильные конкуренты на runtime-слое

### LangGraph

- state graph + conditional routing
- checkpoints на supersteps
- interrupt / resume / time travel
- durable queue workers в Agent Server

Граф хранит состояние приложения; agents часто являются nodes/tools.

### Microsoft Agent Framework

- typed executors + edges
- Sequential / Concurrent / Handoff / Group Chat / Magentic
- BSP superstep execution
- Durable Task: sessions, recovery, distributed workers

Очень близко к typed workflow runtime; global protocol не отдельный artifact.

Reagent сегодня проигрывает обоим в crash recovery, packaging и operational maturity.

# Dapr Agents: сильный runtime без protocol DSL

## Что уже решено

- DurableAgent поверх Dapr Workflows
- checkpointed LLM и tool interactions
- retries, recovery, deterministic execution
- virtual-actor identity and state
- independent services + Pub/Sub

## Что остаётся code-first

- Dapr workflows and child calls
- message-router handlers
- topic and service topology
- нет global role/message choreography
- нет compiler-generated local projections

Dapr закрывает runtime-часть обещания Reagent лучше, чем Reagent сегодня. Отстройка остаётся только на protocol semantics.

# AutoGen и Google ADK

## Богатый словарь orchestration patterns

### AutoGen

#### **GraphFlow**

edges, parallel, conditions, loops

#### **SelectorGroupChat**

model picks next speaker

#### **Swarm**

decentralized handoffs

#### **Core runtime**

lifecycle + messaging

### Google ADK

#### **Graph workflows**

nodes, edges, routes, joins

#### **Workflow agents**

sequential, parallel, loop

#### **Dynamic workflows**

programmatic orchestration

#### **Resume**

completed nodes restored; idempotency required

AutoGen описывает team dynamics. ADK — graph/composition runtime. Оба исполняют swarm, но не требуют global message choreography.

# CrewAI: роли есть, protocol — нет

## Crews

Agents + roles + tasks + process

**Sequential** — фиксированный порядок

**Hierarchical** — manager delegates and validates

## Flows

Event-driven application workflow

routing · conditions · state · persistence · resume/fork

CrewAI сам рекомендует для production начинать с **Flow** и встраивать Crew внутрь.

Следствие: production abstraction снова становится workflow вокруг agent team.

# Swarm frameworks: SOP, planning, conversation

## MetaGPT

Code = SOP(Team)

Roles watch upstream message/action types.

## ChatDev

ChatChain + phases

JSON-configured role dialogues for software process.

## CAMEL

Workforce

Planner decomposes and coordinator assigns workers.

## AgentScope

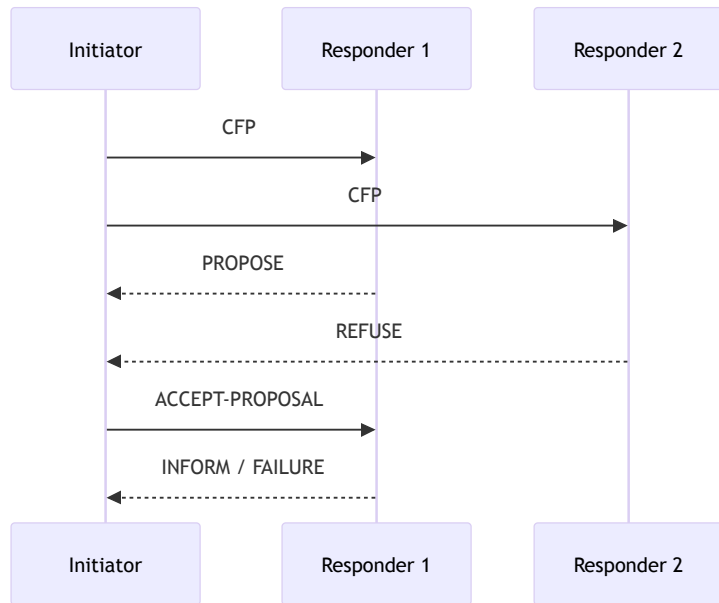
Pipelines + MsgHub

Sequential/fanout execution and broadcast context.

Они доказывают спрос на **structured collaboration**.

Но scenario semantics остаётся в Python/config/prompts и редко становится independently verifiable contract.

# FIPA → Reagent: protocol-first идея для современных агентов



## ACL semantics

performative, conversation-id, ontology

## Role behaviours

initiator / responder state machines

## Distributed runtime

containers, AMS, DF, transports

Sources: JADE Technical Description · jade.proto · ContractNetInitiator

# JaCaMo + Moise: организация как программа

## Structural

roles · groups · links · cardinalities

## Functional

goals · missions · schemes ·  
sequence/choice/parallel

## Normative

obligations · permissions · deadlines

Отдельная organization specification используется и агентами, и runtime-ом, который её enforce.

Moise ближе к «constitutional layer». Reagent — к «conversation choreography».

# Сравнение современных runtime-ов

| Platform                  | Scenario descriptor                          | Durable resume | HITL | Runtime shape                 |
|---------------------------|----------------------------------------------|----------------|------|-------------------------------|
| <b>LangGraph</b>          | state graph / conditional edges              | ●●●            | ●●●  | graph workers + Agent Server  |
| <b>MS Agent Framework</b> | typed executors / edges / orchestrations     | ●●●            | ●●●  | superstep workflow runtime    |
| <b>Dapr Agents</b>        | durable workflows / message routers / pubsub | ●●●            | ●●   | actors + distributed services |
| <b>AutoGen</b>            | GraphFlow / team policy / handoffs           | ●              | ●●   | AgentChat + Core runtime      |
| <b>Google ADK</b>         | workflow-agent composition                   | ●●             | ●●   | Runner + session services     |
| <b>CrewAI</b>             | Crew process + event-driven Flow             | ●●             | ●●   | Flow runtime + Crews          |

●●● = first-class documented capability · ●● = available with explicit setup · ● = limited/core-adjacent. Qualitative snapshot, not benchmark.

Branching, parallelism и handoffs уже commodity. Reagent нельзя позиционировать через этот feature list.

# Где конкуренты сильнее Reagent сегодня

## **Durable execution**

checkpoint/resume на уровне шага, а не только process record

## **Runtime trust**

стабильные failure semantics и production storage

## **Packaging**

installable SDK, version policy, hosted deployment

## **Ecosystem**

models, tools, connectors, cloud integrations

## **Typed enforcement**

MSAF validates edge compatibility; Reagent message schemas пока documentary

## **Operational surface**

queues, workers, auth, managed observability

Ширина архитектуры Reagent уже большая. Доверие к guarantees пока меньше.

# Где остаётся пространство для Reagent

## Standalone protocol artifact

Coordination versioned отдельно от agent code и prompts.

## Global choreography

Видно, кто кому что может отправить и в какой фазе.

## Role-local projection

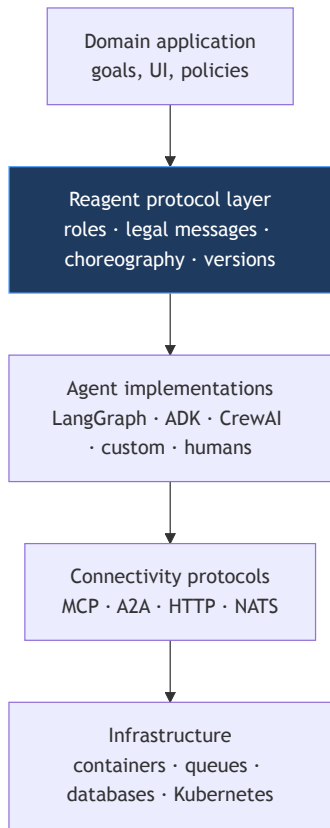
Один protocol компилируется в локальное поведение участников.

## Cross-runtime enforcement

Managed, custom, gate и MCP agents под одним interaction contract.

Дифференциация — не «мы тоже запускаем граф», а **protocol is the product boundary**.

# Reagent должен занять отдельный слой



# Что можно и нельзя заявлять

## Defensible

- protocol — отдельный coordination artifact
- global choreography, не только app graph
- одна модель для heterogeneous participants
- путь к independent audit / conformance / verification
- wire-agnostic interaction layer

## Не заявлять

- «конкурентов нет»
- «никто не описывает multi-agent workflows»
- «первые исполняемые interaction protocols»
- «production durability уже решена»
- «typed/formal guarantees покрывают host code»

Честная отстройка сильнее широкой претензии на уникальность.

# Стратегический вывод

1

## **Сузить wedge**

TS-first approval-gated workflows  
для trusted agent teams.

2

## **Усилить contract**

Typed messages, role projection  
conformance, branch semantics.

3

## **Догнать runtime**

Checkpoint/resume, CI, packaging,  
auth, one golden path.

Новые hosts и transports — после того, как protocol guarantees стали сильнее обычного workflow graph.

# Рынок уже умеет запускать swarm

Открытый вопрос — кто сможет сделать его взаимодействие

**контрактом, а не побочным эффектом кода**

Primary sources and full matrix: `competitors/files/research.md`